

EXERCÍCIOS DE APLICAÇÃO

1. Desenvolva um *script* modularizado para validação de alvos de rede. Crie uma função que valide se um endereço de IP se encontra no formato correto (quatro blocos numéricos separados por pontos) e outra função que valide se uma porta de rede se encontra no intervalo permitido (1 a 65535). Aplique *Type Hinting* explícito em todos os parâmetros e retornos. Construa uma função `main()` que orquestre a chamada destas duas funções e garanta que o *script* apenas é executado se invocado diretamente, utilizando o bloco `if __name__ == "__main__":`.
2. Construa uma função que receba uma lista de *logs* de tráfego (cadeias de caracteres) e devolva uma nova lista contendo apenas os endereços IP externos, ignorando endereços locais (iniciados por "127." ou "192.168."). A função não tem autorização para alterar a lista mutável original recebida por parâmetro. No bloco `main()`, imprima a lista original e a lista filtrada para provar a ausência de contaminação cruzada de dados.
3. Crie um *script* que simula o arranque de uma ferramenta de auditoria. A função `main()` deve recolher uma porta alvo passada obrigatoriamente através da linha de comandos (`sys.argv[1]`). Utilize um bloco `try-except` para capturar uma anomalia de conversão (`ValueError`) caso o utilizador insira letras em vez de números. Ocorrendo a falha, o processo deve ser interrompido imediatamente utilizando `sys.exit(1)` e emitindo um alerta limpo no ecrã. Caso a conversão seja bem-sucedida, imprima o estado de arranque e encerre com `sys.exit(0)`.
4. Implemente uma função que simula a abertura de um *socket* de rede (utilizando a impressão da mensagem "Ligação de rede estabelecida"). Dentro do bloco `try`, provoque intencionalmente um erro matemático (como uma divisão por zero durante o cálculo de latência). Aplique a cláusula `finally` para garantir que a instrução "Ligação de rede encerrada e recursos libertados" é executada de forma incondicional, prevenindo a retenção de processos em memória, independentemente da falha provocada.
5. Desenvolva uma função matemática denominada `calculate_payload_size` que recebe uma cadeia de caracteres e devolve o seu tamanho em *bytes*. Se a cadeia recebida estiver vazia, a função deve levantar um `ValueError` através da instrução `raise`. A função não pode conter qualquer bloco `try-except`. A captura da exceção (Propagação de Erros) tem de ser efetuada no bloco `main()`, onde a função será invocada.

6. Crie uma função responsável por adicionar IPs a uma *allowlist* dinâmica, com a seguinte assinatura: `add_trusted_ip(ip: str, trusted_list: Optional[list] = None) -> list`. Configure a inicialização segura da lista no interior da função para evitar a partilha de memória. No bloco `main()`, invoque a função duas vezes de forma independente, sem passar a lista por parâmetro. Imprima os resultados para provar que a primeira lista não transferiu os seus dados para a segunda execução.
7. Construa um orquestrador para um *scanner* de rede. O `main()` deve capturar dois argumentos via `sys.argv`: IP e Protocolo ("TCP" ou "UDP"). Implemente validações estritas: se o número de argumentos for inválido, levante um `IndexError`; se o protocolo não for estritamente "TCP" ou "UDP", levante um `ValueError`. O bloco `try-except` central do `main()` deve intercepar estas duas exceções distintas, abortando com `sys.exit(1)` após substituir o *Stack Trace* por indicações operacionais legíveis. O bloco `finally` deve registar o fim da tentativa de execução da ferramenta.
8. Desenvolva um motor de *parsing* tático para eventos de segurança. A função `parse_event(log_line: str) -> dict` deve fragmentar a cadeia pelo separador : (esperando o formato IP:AÇÃO). Se a fragmentação não resultar em exatamente duas peças, a função deve gerar um `ValueError`. No `main()`, crie uma lista com cinco eventos (três bem formatados e dois corrompidos). Itere sobre a lista executando o *parsing* dentro de um `try-except`. Se uma linha falhar, o sistema deve imprimir o aviso da anomalia, capturar a exceção e continuar ininterruptamente a análise dos restantes elementos do ciclo sem abortar a operação.